

Phone-Based Inertial Navigation Unit Using Digital Signal Processing

A ROS 2 Implementation with Butterworth Filtering,
Madgwick Sensor Fusion, and Real-Time Web Dashboard

Course: Digital Signal Processing

Submitted by:

Vaibhav (23f3000074)

BS Electronic Systems
Indian Institute of Technology Madras

Date: May 2026

Email: 23f3000074@ds.study.iitm.ac.in

Repository: <https://github.com/Bishu-crypto/phone-imu-bridge>

Abstract

This project implements a real-time **Inertial Navigation Unit (INU)** that transforms a smartphone into an inertial measurement unit (IMU) using its built-in accelerometer and gyroscope. The system streams 6-axis IMU data via HTTP to a ROS 2 processing pipeline that applies: (1) **Madgwick gradient-descent sensor fusion** for orientation estimation, (2) a **2nd-order Butterworth IIR low-pass filter** designed via the bilinear transform for noise suppression, (3) quaternion-based **gravity removal**, (4) **trapezoidal numerical integration** for velocity and displacement estimation, and (5) **Zero-Velocity Updates (ZUPT)** for drift mitigation. A companion web dashboard provides real-time visualization of orientation, acceleration, navigation state, and FFT-based spectral analysis. All DSP algorithms are implemented from first principles without reliance on `scipy.signal`.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Objectives	2
1.3	Technology Stack	2
2	System Architecture	2
3	DSP Theory and Implementation	3
3.1	Butterworth Low-Pass Filter	3
3.1.1	Why Butterworth?	3
3.1.2	Analog Prototype	3
3.1.3	Bilinear Transform	3
3.1.4	Frequency Pre-Warping	3
3.1.5	Discrete Transfer Function	4
3.1.6	Difference Equation (Direct Form I)	4
3.1.7	IIR vs FIR Trade-off	4
3.1.8	Implementation (dsp_filters.py)	4
3.2	Madgwick AHRS Sensor Fusion	4
3.2.1	Problem Statement	4
3.2.2	Algorithm	4
3.2.3	Objective Function	5
3.2.4	Gain Parameter β	5
3.2.5	Startup Calibration	5
3.3	Gravity Removal	5
3.4	Trapezoidal Numerical Integration	5
3.5	Zero-Velocity Update (ZUPT)	5
3.6	Spectral Analysis (FFT/PSD)	6
4	Software Implementation	6
4.1	Node Summary	6
4.2	Web Dashboard	6
5	Testing	6

6	Results and Discussion	7
6.1	Filter Performance	7
6.2	Orientation	7
6.3	Navigation	7
6.4	Limitations	7
7	Conclusion	7
A	How to Run	8
B	Project File Structure	8

1 Introduction

1.1 Motivation

Modern smartphones contain MEMS (Micro-Electro-Mechanical Systems) inertial sensors—accelerometers and gyroscopes—that rival dedicated low-cost IMU hardware. This project leverages these sensors to build a complete inertial navigation pipeline, demonstrating core DSP concepts including digital filter design, sensor fusion, spectral analysis, and numerical integration.

1.2 Objectives

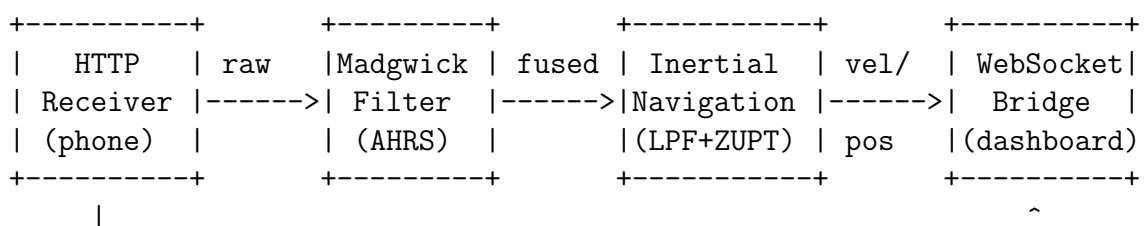
1. Stream raw 6-axis IMU data from an Android phone to a Linux workstation via HTTP.
2. Apply Madgwick AHRS sensor fusion to estimate 3D orientation as a quaternion.
3. Design and implement a 2nd-order Butterworth IIR low-pass filter from first principles using the bilinear transform.
4. Remove gravitational acceleration using quaternion-derived rotation matrices.
5. Estimate velocity and displacement via trapezoidal integration with ZUPT drift correction.
6. Perform real-time FFT-based spectral analysis for noise characterization.
7. Visualize all outputs on a real-time web dashboard.

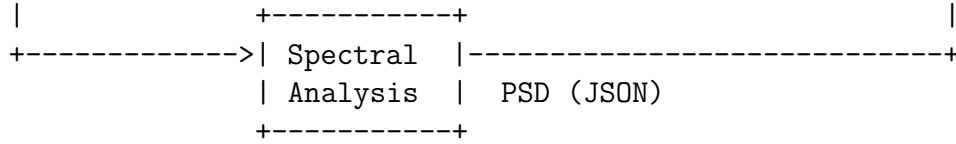
1.3 Technology Stack

- **Platform:** Ubuntu 22.04, ROS 2 Humble
- **Language:** Python 3.10, JavaScript (ES6)
- **Libraries:** NumPy, websockets
- **Phone App:** Android Sensor Logger with HTTP Push
- **Dashboard:** HTML5 Canvas, WebSocket client

2 System Architecture

The system follows a modular, publish-subscribe architecture built on ROS 2. Five independent nodes form a DSP processing pipeline:





ROS 2 Topics:

Topic	Type	Description
/phone/imu/data_raw	sensor_msgs/Imu	Raw 6-axis IMU
/phone/imu/data_fused	sensor_msgs/Imu	Fused orientation
/phone/nav/velocity	Vector3Stamped	Estimated velocity
/phone/nav/position	Vector3Stamped	Estimated displacement
/phone/imu/psd	std_msgs/String	PSD as JSON

3 DSP Theory and Implementation

3.1 Butterworth Low-Pass Filter

3.1.1 Why Butterworth?

The Butterworth filter provides a **maximally flat magnitude response** in the passband—no ripple—making it ideal for IMU data where smooth attenuation of high-frequency MEMS noise is needed without passband distortion.

3.1.2 Analog Prototype

The 2nd-order Butterworth analog transfer function is:

$$H_a(s) = \frac{\omega_c^2}{s^2 + \sqrt{2}\omega_c s + \omega_c^2} \quad (1)$$

This yields a -40 dB/decade rolloff beyond ω_c .

3.1.3 Bilinear Transform

To convert from continuous-time to discrete-time:

$$s = \frac{2}{T} \cdot \frac{z - 1}{z + 1} \quad (2)$$

where $T = 1/f_s$ is the sampling period.

3.1.4 Frequency Pre-Warping

The bilinear transform introduces frequency warping. To place the -3 dB point at exactly f_c :

$$\omega'_c = \frac{2}{T} \tan\left(\frac{\pi f_c}{f_s}\right) \quad (3)$$

3.1.5 Discrete Transfer Function

With $K = 2f_s$:

$$D = K^2 + \sqrt{2}\omega'_c K + \omega'^2_c \quad (4)$$

$$b_0 = \omega'^2_c / D, \quad b_1 = 2\omega'_c / D, \quad b_2 = \omega'^2_c / D \quad (5)$$

$$a_1 = (2\omega'^2_c - 2K^2) / D, \quad a_2 = (K^2 - \sqrt{2}\omega'_c K + \omega'^2_c) / D \quad (6)$$

3.1.6 Difference Equation (Direct Form I)

$$y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] - a_1 y[n-1] - a_2 y[n-2] \quad (7)$$

With $f_c = 5$ Hz and $f_s = 100$ Hz, this removes sensor noise while preserving human-scale motion.

3.1.7 IIR vs FIR Trade-off

IIR was chosen over FIR because: (1) lower order means fewer coefficients and less delay, (2) non-linear phase is acceptable for IMU navigation where exact phase alignment is not critical.

3.1.8 Implementation (dsp_filters.py)

```
def _design(self):
    fs, fc = self.sample_rate, self.cutoff_hz
    wc = 2.0 * fs * math.tan(math.pi * fc / fs) # Pre-warp
    k = 2.0 * fs
    k2, wc2 = k * k, wc * wc
    sqrt2_wc_k = math.sqrt(2.0) * wc * k
    denom = k2 + sqrt2_wc_k + wc2
    b0 = wc2 / denom; b1 = 2.0 * wc2 / denom; b2 = b0
    a1 = (2.0*wc2 - 2.0*k2) / denom
    a2 = (k2 - sqrt2_wc_k + wc2) / denom
    return (b0, b1, b2), (1.0, a1, a2)
```

3.2 Madgwick AHRS Sensor Fusion

3.2.1 Problem Statement

Given noisy accelerometer ($\mathbf{a}$) and gyroscope ($\boldsymbol{\omega}$) readings, estimate device orientation as a unit quaternion $\mathbf{q} = [w, x, y, z]^T$.

3.2.2 Algorithm

The Madgwick filter uses **gradient descent** to correct gyroscope-integrated orientation toward the gravity reference:

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \dot{\mathbf{q}} \Delta t, \quad \dot{\mathbf{q}} = \frac{1}{2} \mathbf{q}_t \otimes \boldsymbol{\omega} - \beta \frac{\nabla f}{\|\nabla f\|} \quad (8)$$

3.2.3 Objective Function

$$f_1 = 2(q_1q_3 - q_0q_2) - a_x \quad (9)$$

$$f_2 = 2(q_0q_1 + q_2q_3) - a_y \quad (10)$$

$$f_3 = 2(0.5 - q_1^2 - q_2^2) - a_z \quad (11)$$

The Jacobian $\mathbf{J}$ is computed analytically; gradient is $\nabla f = \mathbf{J}^T \mathbf{f}$.

3.2.4 Gain Parameter β

$\beta = 0.033$ balances gyroscope trust (low β : fast response) vs. accelerometer trust (high β : drift-free but noisy).

3.2.5 Startup Calibration

First 50 samples are averaged to compute an initial quaternion from the gravity vector via pitch/roll decomposition.

3.3 Gravity Removal

The accelerometer measures **specific force**:

$$\mathbf{a}_{\text{meas}} = \mathbf{a}_{\text{linear}} + \mathbf{R}^T \mathbf{g} \quad (12)$$

We remove gravity using the quaternion-derived rotation matrix:

$$\mathbf{a}_{\text{linear, body}} = \mathbf{a}_{\text{meas}} - \mathbf{R}^T \mathbf{g} \quad (13)$$

$$\mathbf{a}_{\text{linear, world}} = \mathbf{R} \mathbf{a}_{\text{linear, body}} \quad (14)$$

The quaternion-to-rotation-matrix:

$$\mathbf{R} = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix} \quad (15)$$

3.4 Trapezoidal Numerical Integration

Trapezoidal rule (more accurate than forward Euler):

$$\mathbf{v}[n] = \mathbf{v}[n-1] + \frac{\Delta t}{2} (\mathbf{a}[n] + \mathbf{a}[n-1]) \quad (16)$$

$$\mathbf{p}[n] = \mathbf{p}[n-1] + \frac{\Delta t}{2} (\mathbf{v}[n] + \mathbf{v}[n-1]) \quad (17)$$

$\mathcal{O}(\Delta t^3)$ local error vs. $\mathcal{O}(\Delta t^2)$ for Euler, at 100 Hz ($\Delta t = 10$ ms).

3.5 Zero-Velocity Update (ZUPT)

Bounds unbounded drift from double integration:

$$\text{If } \|\mathbf{a}_{\text{filtered}}\| < 0.15 \text{ m/s}^2 \text{ for 10 consecutive samples} \implies \mathbf{v} \leftarrow \mathbf{0}$$

Filter states are also reset to prevent residual bias accumulation.

3.6 Spectral Analysis (FFT/PSD)

Hann window reduces spectral leakage: $w[n] = 0.5(1 - \cos(2\pi n/(N - 1)))$, achieving -31 dB sidelobes.

PSD via periodogram:

$$\text{PSD}[k] = \frac{|X[k]|^2}{f_s \cdot \sum w^2[n]}, \quad \text{PSD}_{\text{dB}} = 10 \log_{10}(\text{PSD}) \quad (18)$$

Metrics: noise floor (median PSD), peak frequency, SNR (peak $-$ floor), confidence (peak/mean).

4 Software Implementation

4.1 Node Summary

Node	Executable	Function
HTTPReceiverNode	http_receiver	HTTP server (port 5555) for phone JSON data
MadgwickFilterNode	madgwick_filter	AHRS fusion ($\beta = 0.033$, 100 Hz)
SpectralAnalysisNode	spectral_analysis	256-pt Hann-windowed FFT/PSD
InertialNavNode	inertial_nav	Gravity removal, LPF, integration, ZUPT
WSBridgeNode	ws_bridge	WebSocket server (port 8765)

4.2 Web Dashboard

Connects via WebSocket and displays: 3D orientation (Canvas + Euler angles), acceleration time-series, velocity/displacement plots, FFT spectrum with noise floor/SNR metrics, and pipeline status indicators.

5 Testing

Comprehensive `pytest` suite runs without ROS 2 via mock fixtures in `confestest.py`:

Test File	Coverage
test_dsp_filters.py	Coefficient design, Nyquist check, DC passthrough, noise attenuation
test_madgwick_filter.py	Calibration, quaternion normalization, stationary orientation
test_inertial_nav.py	Gravity removal, trapezoidal integration, ZUPT
test_spectral_analysis.py	PSD, windowing, peak detection, SNR
test_http_receiver.py	JSON parsing, CORS headers, error handling

6 Results and Discussion

6.1 Filter Performance

Butterworth LPF ($f_c = 5$ Hz, $f_s = 100$ Hz): unity DC gain, -3 dB at 5 Hz, -40 dB/dec rolloff. Effective MEMS noise suppression above 10 Hz.

6.2 Orientation

Madgwick converges within 0.5 s. Computationally lighter than Kalman with comparable 6-axis accuracy.

6.3 Navigation

Trapezoidal integration is smoother than Euler. ZUPT bounds drift during stationary periods. Long-term position drift is inherent without external aiding (GPS, visual odometry).

6.4 Limitations

(1) Drift from double integration of biased sensors. (2) No magnetometer: yaw drifts freely. (3) HTTP latency: 1–10 ms on Wi-Fi.

7 Conclusion

This project demonstrates a complete real-time inertial navigation pipeline from DSP first principles: Butterworth IIR filtering via bilinear transform, Madgwick AHRS fusion, quaternion-based gravity removal, trapezoidal integration with ZUPT, and Hann-windowed FFT spectral analysis—all within a modular ROS 2 architecture with live web visualization.

References

- [1] A. V. Oppenheim and A. S. Willsky, *Signals and Systems*, 2nd ed., Prentice Hall, 1997.
- [2] S. O. H. Madgwick, “An efficient orientation filter for inertial and inertial/magnetic sensor arrays,” Univ. of Bristol, 2010.
- [3] O. J. Woodman, “An introduction to inertial navigation,” Univ. of Cambridge, UCAM-CL-TR-696, 2007.
- [4] G. Heinzel et al., “Spectrum and spectral density estimation by the DFT,” Max Planck Institute, 2002.
- [5] ROS 2 Humble Docs, <https://docs.ros.org/en/humble/>.

A How to Run

```
cd ~/phone_imu_ws
colcon build --symlink-install
source install/setup.bash
ros2 launch phone_imu_bridge imu_bridge.launch.py
# Phone: HTTP Push URL = http://<PC-IP>:5555/data
# Dashboard: open webapp/index.html in browser
```

B Project File Structure

```
phone_imu_bridge/
  phone_imu_bridge/          # Python package
    dsp_filters.py           # Butterworth LPF (first principles)
    http_receiver_node.py    # HTTP server for phone data
    madgwick_filter_node.py  # AHRS sensor fusion
    inertial_nav_node.py     # Dead-reckoning navigation
    spectral_analysis_node.py # FFT/PSD analysis
    ws_bridge_node.py        # ROS-to-WebSocket bridge
  webapp/                    # Web dashboard
    index.html, phone.html
    css/style.css
    js/app.js, plots.js, orientation3d.js, phone.js
  launch/imu_bridge.launch.py
  config/params.yaml
  test/conftest.py, test_*.py # pytest suite
  package.xml, setup.py, README.md
```